PITCH DECK · V0.1.1 · MIT LICENSE

lia.

A local-first AI assistant written in Rust.
One agent core, many ways to reach it.

Terminal, web, voice, and chat platforms — running on your machine, working inside your workspace, and asking before it acts. Conversations, memory and skills stay on your disk.

RUST 2024 EDITION · ~43,300 LINES · 86 MODULES · 35 INTEGRATION TESTS · 4 CHAT PLATFORMS · 4 PROVIDERS · LOCAL MODEL VIA OLLAMA

Runs on your machine. You choose the model — including fully local ones — and the assistant never leaves your disk. No remote ownership of your data, no single vendor lock-in.

AI assistants today fall into two camps, neither of which fits real work.

CAMP ONE — CLOUD CHATBOTS

- **own your data.** Every prompt leaves the machine and stays on someone else's server.
- **cannot touch your files.** They describe the change; you do it by hand.
- **forget between sessions.** No durable memory, no audit trail of what was tried.

CAMP TWO — CODING AGENTS

- **send your code out.** Powerful, but your source travels to a remote service.
- **risk without consent.** Shell and file changes happen with little or no approval.
- **scatter.** Memory in one app, voice in another, scheduling in a third, chat in a fourth.

lia exists so the assistant lives where the work is: on your disk, in your workspace, under your control. Nothing persists in someone else's cloud; nothing acts without asking.

One agent. Every surface. Fully local by default.

lia is a single Rust binary. It owns the agent loop, the tools, memory, skills, scheduling, and voice. You reach that one core from wherever you are:

- **Terminal (TUI)** — ratatui interface for focused coding work, sessions and command palette.
- **Browser** — http://127.0.0.1:8787 visual deck with a system monitor and conversation orb.
- **Voice** — microphone in, speech out, mic muted while it talks; hands-free wake word "lia".
- **Chat platforms** — Telegram, Discord, Slack, and WhatsApp linked as a device.
- **Gateway** — HTTP and WebSocket so any program can drive the same agent.
- **Embedded web** — styles and scripts served under /assets/ via include_str!; UI works offline.
- **Workspace-aware** — the assistant works in your repo, never elsewhere on disk.
- **Model choice** — Anthropic, OpenAI-compatible, Ollama, llama.cpp; you decide.

One core, many doors. Conversations, memory and skills stay on your disk regardless of which surface you reach it through. Switch from terminal to browser to phone without losing the thread.

The reasons it earns a place in your daily work.

LOCAL-FIRST — DISK

Conversations, memory and skills live on your disk. You choose the model, including local ones through Ollama.

DOES THE WORK — TOOLS

Reads and edits files, runs commands, searches the web, reads screenshots, and reports exactly what it touched.

SAFE BY DEFAULT — ASK

Commands need your approval and stay in your workspace. Strangers on chat platforms are paired with a code you approve.

RESILIENT — FALLBACK

If a provider fails or runs out of credit, the next one in your fallback list takes over.

ONE AGENT — SURFACES

The gateway exposes the same agent over HTTP and WebSocket, with adapters for Telegram, Discord, Slack and WhatsApp.

EXTENSIBLE — SKILLS/MCP

Add procedures as skills and borrow tools from any MCP server alongside the built-in ones.

What you get.

AREA	WHAT YOU GET
Coding	File reading and editing, sandboxed command execution, plans ticked off as work proceeds.
Memory	Saved sessions, daily notes consolidated into durable facts, each tagged with its source.
Skills	Written procedures loaded on demand: security audit, release checklist, incident triage, and more.
Multimodal	Pictures in, voice in and out.
Web	Search and page reading.
Automation	Scheduled work: "check the build every half hour", answered in the chat that asked.
Delegation	Heavy jobs handed to a sub-assistant so the conversation stays clean.
Providers	Anthropic, OpenAI-compatible, Ollama, llama.cpp, with retry and fallback.

Concrete capabilities, not aspirations.

- **Reads and edits your code.** Opens the file, changes it, tells you what it touched.
- **Runs commands** with your approval, confined to your workspace.
- **Remembers.** Conversations survive a restart; daily notes sorted into what's worth keeping.
- **Learns the job.** A skill is a written procedure loaded when a task needs one.
- **Borrows other tools.** MCP servers offer their tools alongside built-in ones.
- **Looks things up.** Searches the web, reads the page, reports what's there.
- **Sees pictures.** Screenshots or photographs, in chat, terminal, or over the gateway.
- **Hears and speaks.** Voice command in, spoken answer out, mic shut while talking.
- **Keeps a plan.** Multi-step work written down and ticked off.
- **Hands work over.** Heavy jobs go to another assistant; the answer comes back.
- **Keeps its promises.** Scheduled jobs kept and answered where asked.
- **Keeps answering.** When a model fails, the next one in the list is asked instead.

A modular Rust workspace, one binary, assets inside.

PATH	RESPONSIBILITY
src/agent/	The loop, session storage, context compaction.
src/provider/	Neutral message types plus Anthropic, OpenAI, Ollama and llama.cpp adapters.
src/tool/	Built-in tools and the approval policy.
src/mcp/	MCP client and the file server it starts for itself.
src/gateway/	The daemon: HTTP and WebSocket.
src/channel/	Messaging platform adapters.
src/voice/	Microphone, speech recognition, and speaker.
src/tui/	Terminal interface.
src/schedule/	Work the assistant promises to do later.
web/	Browser interface, one file per region of the page.

Stylesheets and scripts are served under **/assets/** and embedded via **include_str!**, so the UI works offline alongside the single binary.

Built to be trusted with your machine.

- **The workspace.** Every file tool resolves against the workspace root and refuses to step outside it, symlinks included.
- **Approval.** Tools that change the system ask first. Policy is ask, allow, or deny. One-shot runs deny risky tools by default.
- **Blocked commands.** The shell tool refuses destructive patterns outright, whatever the policy says.
- **A sandbox where there is one.** Commands can run under bubblewrap: the workspace is the one writable place.
- **Provenance.** A remembered fact is only as trusted as where it came from. Network content is data, never instruction.
- **Credentials.** Keys live in the environment, not the config file. The gateway binds to 127.0.0.1 and can require a bearer token.
- **Pictures.** Chat images are fetched from the platform's own servers only, with no redirects.

Same agent, every door.

SURFACE	NOTES
Terminal	ratatui TUI with sessions, composing and command palette.
Browser	Deck UI: system monitor, conversation orb, command console, drawers.
Voice	Mic in, speech out, hands-free mode listening for "lia".
Telegram	Long-polling bot with pairing codes for strangers.
Discord	REST API adapter with channel and user allow-lists.
Slack	Web API adapter, same model as Discord.
WhatsApp	Links as a WhatsApp Web device, scanned once via QR.
Gateway	HTTP and WebSocket: /v1/sessions, /v1/inbox, health, audio, system.

Model access is provider-neutral, built to survive outages.

- Supports Anthropic, OpenAI-compatible endpoints, Ollama and llama.cpp.
- Retry on transient failures with backoff.
- Fallback list: when one provider fails or runs out of credit, the next is tried.
- Streaming replies with cancellation.
- Capability discovery via /v1/models and /v1/tools.
- You are never locked into one vendor, and a failing model does not take the whole task down.

ANTHROPIC FIRST-CLASS | OPENAI COMPATIBLE | OLLAMA LOCAL | LLAMA.CPP LOCAL | RETRY + BACKOFF | FALLBACK LIST | STREAMING + CANCEL

The assistant remembers, learns, and follows through.

- Saved sessions survive a restart and can be resumed in any surface.
- Daily notes consolidate into durable facts, each tagged with its source.
- Skills are procedures on disk, loaded only when a task needs them.
- Scheduled work is kept, retried, and answered in the chat that asked for it.
- Heavy jobs are delegated to a sub-assistant so the main conversation stays clean.
- MCP servers lend their tools, offered alongside the built-in ones.

Beyond coding: lia is becoming a daily co-pilot.

FEATURE	STATUS
Local inbox, tasks and notes	Implemented (CLI, gateway routes, web panel).
Timezone-aware routines	Implemented.
Briefings (morning, end of day, weekly)	Implemented.
Read-only calendar (CalDAV)	Implemented.
Follow-ups with reminders	Implemented.
Read-only email triage (JMAP)	Implemented.
Approval-gated mail sending	Implemented (F12 gate).
Agent-initiated writes	Deliberately not done.

All personal features are opt-in, local by default, and treat network content as untrusted.

Solid, shipped, growing.

LANGUAGE **Rust** 2024	SOURCE **~43,300** lines	MODULES **86**	TESTS **35** integration
WEB **30** frontend files	CHANNELS **4**	PROVIDERS **4**	LICENSE **MIT · v0.1.1**

Contributions welcome. Start with the **Development** and **Roadmap** docs for where help is most useful.

Dependability over novelty.

Proposed improvements, not a release schedule — focused on making existing capabilities more dependable, observable and easier to extend.

- **Automated checks.** CI for formatting, linting, tests, web build.
- **Behavioral coverage.** Streaming, approval, persistence, gateway auth.
- **Consistent failures.** Stable error taxonomy, retryable categories.
- **Provider behavior.** Explicit capability selection, mid-stream failure handling.
- **Context management.** Visible compaction, token limits, overflow recovery.
- **Memory quality.** Duplicate handling, correction controls, optional semantic retrieval.
- **Skills lifecycle.** Validation, dependencies, safe import and export.
- **Daily co-pilot.** Inbox, routines, briefings, calendar, follow-ups, email triage.

Get started in three commands.

```
$ git clone https://github.com/binkapS/lia.git
$ cd lia && ./install.sh
$ lia --help
```

Or manually: npm install && npm run build:web, then cargo build --release. Copy lia.example.toml as a starting configuration. See docs/install.md and docs/configure.md.

lia runs on your machine, works inside your workspace, and asks before it acts.
Local-first · Safe by default · Resilient · One agent, every surface · Extensible.